560. Subarray Sum Equals K

At every index `i`, count how many past prefix sums equal `prefix_sum[i] - k`, because each one marks the start of a valid subarray **ending at** `i`.

ie: If `diff` was seen at index `j`, then the subarray from `j+1` **to** `i` has sum = k.

> 💡 Count the subarray from `j+1` **to** `i` that has a sum = k.

Example : `[3, 4, 7, 2, -3, 1, 4, 3, 1]`, k=7

prefix sum array:

i	nums[i]	prefix_sum
0	3	3
1	4	7
2	7	14
3	2	16
4	-3	13
5	1	14
6	4	18
7	3	21
8	1	22

At `i=7`, `diff=14`, and `dic[14]` was seen at `j=2` and `j=5`.

```
j=2 → subarray from j+1=3 to i=7 = [2, -3, 1, 4, 3] ✅
j=5 → subarray from j+1=6 to i=7 = [4, 3]           ✅
```

So we count the subarray up until index i.

The Big Idea

`prefix_sum[i] - prefix_sum[j] = k` means the subarray from `j+1` to `i` has sum = k.

So when we're at index `i`, we ask: **"how many previous prefix sums equal `prefix_sum[i] - k`?"**

Prefix Sums

`nums = [3, 4, 7, 2, -3, 1, 4, 3, 1]`, k=7

i	nums[i]	prefix_sum
0	3	3
1	4	7
2	7	14
3	2	16
4	-3	13
5	1	14
6	4	18
7	3	21
8	1	22

Walking Through Each Index

i=0, prefix_sum=3, diff=3-7=-4

- dic = `{0:1}`, -4 not found → count=0
- update dic → `{0:1, 3:1}`

i=1, prefix_sum=7, diff=7-7=0

- dic = `{0:1, 3:1}`, 0 found! dic[0]=1 → count=**1**
- That `{0:1},` 1 means: prefix_sum 0 existed at **one place** (the start, before index 0)
- Subarray found: `[3,4]` (from i=0 to i=1) ✅
- update dic → `{0:1, 3:1, 7:1}`

i=2, prefix_sum=14, diff=14-7=7

- dic = `{0:1, 3:1, 7:1}`, 7 found! dic[7]=1 → count=**2**
- That 1 means: prefix_sum 7 existed at **one place** (i=1)
- Subarray found: `[7]` (from i=2 to i=2) ✅
- update dic → `{0:1, 3:1, 7:1, 14:1}`

i=3, prefix_sum=16, diff=16-7=9

- 9 not in dic → count stays 2
- update dic → `{..., 16:1}`

i=4, prefix_sum=13, diff=13-7=6

- 6 not in dic → count stays 2
- update dic → `{..., 13:1}`

i=5, prefix_sum=14, diff=14-7=7

- dic has 7! dic[7]=1 → count=**3**
- Subarray found: `[7, 2, -3, 1]` (from i=2 to i=5) ✅
- update dic → `{..., 14:2}` ← 14 now seen twice!

i=6, prefix_sum=18, diff=18-7=11

- 11 not in dic → count stays 3
- update dic → `{..., 18:1}`

i=7, prefix_sum=21, diff=21-7=14

- dic has 14! **dic[14]=2** → count=**5**
- That **2** means: prefix_sum 14 existed at **two places** (i=2 and i=5)
- Subarray 1: from i=3 to i=7 = `[2,-3,1,4,3]` ✅
- Subarray 2: from i=6 to i=7 = `[4,3]` ✅
- **This is why we do `+=dic[diff]` and not `+=1`** — there were 2 matching prefix sums, so 2 subarrays found at once!

i=8, prefix_sum=22, diff=22-7=15

- 15 not in dic → count stays 5

Final Answer = 5

subarray	found at
`[3,4]`	i=1
`[7]`	i=2
`[7,2,-3,1]`	i=5
`[2,-3,1,4,3]`	i=7
`[4,3]`	i=7

Why `+= dic[diff]`

The reason we do `+= dic[diff]` is because at `i=7`, **two** previous indices had prefix_sum=14, meaning **two different subarrays** both end at `i=7` with sum=7. If we did `+=1` we would have only counted one of them and missed the other.

from our problem

At `i=7`, `diff=14`, and `dic[14]` was seen at `j=2` and `j=5`.

```
j=2 → subarray from j+1=3 to i=7 = [2, -3, 1, 4, 3] ✅
j=5 → subarray from j+1=6 to i=7 = [4, 3]           ✅
```